

Homework 2: More Arrays, 2D Memory Access, and Circular Buffers

Submission Format: Submit all `.c` files and `README.txt` in a zipped folder named `LastName_HW2.zip`. Include clear comments in your code, and answer all questions in `README.txt`.

Overview

This assignment builds on your understanding of arrays and memory in C by moving from simple data transformations to low-level memory access patterns and real-world data structures. You'll work with array manipulation, pointer arithmetic, 2D array indexing, and implement a circular buffer commonly used in embedded sensor systems. Each part emphasizes writing modular, testable C code.

Setup Instructions

We've provided a starter zip file `HW2_starter.zip`, which contains:

- One Arduino sketch file: `Homework2.ino`
- Three skeleton `.c` files: `array_ops.c`, `array_2d.c` and `sensor_buffer.c`
- A helper header: `hwlib.h`
- A `README.txt` file for your written answers

To run your code:

- Open Arduino IDE.
- Open the file `Homework2.ino`.
- Ensure the `.ino`, the three `.c` files, and `hwlib.h` are in the same folder.
- Select your ESP32 board under Tools -> Board and select the correct port.
- Upload to your ESP32 and open the Serial Monitor at 9600 baud.
- As you complete each part, its output will print to the Serial Monitor automatically.

NOTE: You should only edit the `.c` files. Do NOT modify the `.ino` or `.h` files.

Part I: Array Fundamentals and Pointer Access

You've learned how pointers relate to memory and how arrays are laid out in contiguous memory blocks. Now we'll reinforce how to use arrays and pointers interchangeably, while also pushing your understanding of memory access and indexing using arithmetic.

Concepts

Array Traversal

An array like `int arr[5] = {1, 2, 3, 4, 5}` can be accessed with either `arr[i]` or `*(arr + i)`. You'll practice both.

Function Modularity

Each function in embedded systems should do one clear job: compute a result, filter data, or transform memory. This makes debugging and reuse easier.

Returning Multiple Outputs

Since C only allows a single return value, we often pass output pointers to store results or return a count from filtered data.

Tasks

Work in the file: `array_ops.c`

Use `printf()` from `hwlib.h`.

Each section has `// TODO`: comments where you will fill in code.

Step 1 - Write and Call Array Functions

Implement the following functions:

```
int sum_array(const int* arr, int size);
int find_max(const int* arr, int size);
int filter_positive(const int* in, int* out, int size);
```

In `run_array_ops()`:

- Create an integer array with the values 1, -2, 3, -4, 5, and -6. Print out the contents of the original array.
- Call and print results of `sum_array()` and `find_max()`
- Declare `int filtered[6]` and call `filter_positive()`, printing the count and elements copied into filtered.

Step 2 - Pointer Offsets

Add a new function:

```
int find_max_offset(const int* arr, int size, int offset);
```

This function should return the maximum value among every offset-th element in the array, starting from index 0.

For example, with input `arr = {1, -2, 3, -4, 5, -6}` and `offset = 2`, the function should compare every second element and find the maximum value. Use pointer arithmetic instead of array indexing where possible.

In `run_array_ops()`:

- Call this with `offset = 2` and print the result.

Example Output

```
Original array: 1 -2 3 -4 5 -6
Sum = -3
Max = 5
Filtered positives (3): 1 3 5
Max with offset 2: 5
```

Deliverables for Part I

- File: `array_ops.c`
- Answer these questions in `README.txt`:
 - Why use pointer arithmetic instead of array indexing in some cases?
 - How do you return multiple values from a function in C?
 - What challenges arise when skipping elements with pointer arithmetic?

Part II: 2D Arrays and Memory Access

Working with multidimensional arrays helps build your intuition for memory layout in C. This section will deepen your understanding of how 2D arrays are stored and accessed, especially using pointers.

Concepts

2D Arrays in Memory

A 2D array like `int grid[2][3]` is actually a flattened block of memory with 6 integers laid out row-by-row (row-major order).

Pointer Access for 2D Arrays

You can access elements in a 2D array using either `grid[i][j]` or pointer arithmetic:

```
*(*(grid + i) + j)
*(&grid[0][0] + (i * cols + j))
```

Manual Row-Major Indexing

For embedded systems, you may work with flattened buffers, so understanding how to convert 2D coordinates to linear indices is useful.

Tasks

Work in the file: `array_2d.c`

Use `printf()` from `hwlib.h`.

Each section has `// TODO`: comments where you will fill in code.

Step 1 - Declare and Print a 2D Array

- Create a 2D array: `int matrix[2][3] = {{1, 2, 3}, {4, 5, 6}}`
- Print each element using:
 - Standard indexing
 - Flattened pointer
- Print each element's address.

Step 2 - Write `void zero_2d(int* data, int rows, int cols)`

This function should zero out a 2D array passed as a flattened pointer.

- Call `zero_2d((int*)matrix, 2, 3);`
- Print the matrix again using standard indexing to confirm it's zeroed.

Example Output

```
matrix[0][0] = 1 at 0x3FFEAC00
matrix[0][1] = 2 at 0x3FFEAC04
matrix[0][2] = 3 at 0x3FFEAC08
matrix[1][0] = 4 at 0x3FFEAC0C
matrix[1][1] = 5 at 0x3FFEAC10
matrix[1][2] = 6 at 0x3FFEAC14
Zeroed matrix:
0 0 0
0 0 0
```

Deliverables for Part II

- File: `array_2d.c`
- Answer these questions in `README.txt`:
 - How is a 2D array laid out in memory?
 - Why might you flatten a 2D array in embedded code?
 - What's the difference between `matrix[i][j]` and `*(&matrix[0][0] + i * cols + j)`?

Part III: Circular Buffer Implementation

Circular buffers (also called ring buffers) are fixed-size buffers where data is written and read in a circular fashion. When the end of the buffer is reached, it wraps around to the beginning.

Concepts

Circular Buffer Mechanics

A circular buffer uses a fixed-size array with two indices:

- head: where to write the next element
- tail: where to read the next element

When head == tail, the buffer is either empty or full, depending on context. To keep things simple, we will leave one slot empty and treat head == tail as empty.

Wraparound with Modulo

Use (index + 1) % capacity to wrap around the end of the buffer.

Tasks

Work in the file: `sensor_buffer.c`

Use `printf()` from `hwlib.h`.

Each section has `// TODO`: comments where you will fill in code.

Step 1 - Declare the Circular Buffer

Define a `SensorBuffer` struct with:

- A `float buffer[5]`
- Two `int` indices: `head` and `tail`

Write `void init_buffer(SensorBuffer* buf)`

This should:

- Set head and tail to 0.
- Zero out the buffer

Step 2 - Implement Basic Operations

Implement these functions:

```
bool is_empty(SensorBuffer* buf);
bool is_full(SensorBuffer* buf);
bool push(SensorBuffer* buf, float value); // returns false if full
bool pop(SensorBuffer* buf, float* out);   // returns false if empty
void print_buffer(SensorBuffer* buf);
```

- Use modulo arithmetic to manage wraparound.
- `print_buffer()` should print all valid contents in order, from tail to head.

Step 3 - Test Your Buffer

In `run_sensor_buffer()`, demonstrate the buffer:

- Push 5 values (should reject the 5th)
- Print buffer
- Pop 2 values
- Push 2 new values (should succeed)
- Print buffer again

Example Output

```
Pushed 1.0
Pushed 2.0
Pushed 3.0
Pushed 4.0
Buffer full, could not push 5.0
Buffer contents: 1.0 2.0 3.0 4.0
Popped 1.0
Popped 2.0
Pushed 5.0
Pushed 6.0
Buffer contents: 3.0 4.0 5.0 6.0
```

Deliverables for Part III

- File: `sensor_buffer.c`
- Answer these questions in `README.txt`:
 - What's the purpose of the head and tail indices?
 - Why do we leave one slot empty in a circular buffer?
 - What is the benefit of using modulo arithmetic?